MASTER_ARCHITECTURE.md

YourBuy — Master Architecture Document

DATABASE ENGINE (VERY IMPORTANT)

The entire YourBuy project MUST use:

MySQL ONLY

Ignore all older PostgreSQL references from previous documentation.

All implementations must be:

- MySQL-compatible
- Prisma-compatible
- scalable
- production-oriented

Mandatory Prisma datasource:

```
.datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}
```

Environment variable format:

```
DATABASE_URL="mysql://USER:PASSWORD@HOST:PORT/DATABASE"
```

NEVER generate:

- PostgreSQL syntax
- PostgreSQL-only features
- PostgreSQL-specific infrastructure

---

1. PROJECT OVERVIEW

Project Name

YourBuy

Product Type

AI-powered fintech-style simulated trading and investing platform.

Primary Goal

Create a realistic trading ecosystem where users can:

- learn investing
- simulate investing
- understand market behavior
- practice portfolio management
- use AI-driven market intelligence
- experience realistic investing workflows

WITHOUT feeling like they are using a classroom/tutorial website.

---

2. PRODUCT PHILOSOPHY

YourBuy should feel:

- modern
- premium
- startup-grade
- professional
- realistic
- intelligent
- scalable

The platform should balance:

- beginner friendliness
- advanced workflows
- clean UI
- realistic investing experience

- contextual education

---

3. CORE PRODUCT MODULES

Main Systems

Authentication System

- signup
- login
- JWT auth
- refresh tokens
- onboarding flow

Portfolio System

- holdings
- P&L tracking
- asset allocation
- portfolio history
- virtual balance
- transaction history

Market System

- realtime stock prices
- watchlists
- market movers
- sector tracking
- historical charts

Trading System

- simulated buying/selling
- order execution
- order history
- realistic market flows
- F&O simulation
- mutual fund simulation

AI Intelligence System

Called:

YourBuy Intelligence

Features:

- contextual AI analysis
- stock explanations
- news intelligence
- sector impact analysis
- portfolio insights
- educational explanations
- smart alerts

Learning System

Integrated naturally into workflows.

NOT a classroom-style module.

Supports:

- beginner overlays
- adaptive onboarding
- progressive complexity
- AI explanations
- guided hints

Dashboard System

Supports:

- adaptive layouts
- beginner/intermediate/advanced users
- customizable widgets later
- professional information density

---

4. USER LEVEL ARCHITECTURE

Level 1 — Beginner

Users receive:

- simplified onboarding
- contextual explanations
- guided overlays
- educational AI hints
- cleaner workflows

Level 2 — Intermediate/Advanced

Users receive:

- denser information
- reduced guidance
- faster workflows
- advanced analytics
- optional advanced features

Level 3 — Professional Tracking Layer

Separate optional section.

Supports:

- advanced performance tracking
- advanced analytics
- deeper portfolio understanding

---

5. FRONTEND ARCHITECTURE

Frontend Stack

- Next.js
- TypeScript
- Tailwind CSS
- Zustand

- TanStack Query
- Framer Motion
- TradingView Widgets

---

6. FRONTEND PHILOSOPHY

Frontend MUST:

- feel premium
- remain responsive
- avoid clutter
- preserve readability
- balance information density
- support adaptive complexity

---

7. UI/UX PRINCIPLES

Navigation System

Hybrid navigation:

Top Navigation

Contains:

- search
- market overview
- notifications
- AI access
- profile controls

Sidebar / Hamburger

Contains:

- portfolio
- watchlists
- orders
- mutual funds
- F&O
- learning dashboard

- settings

---

8. VISUAL PHILOSOPHY

UI should take inspiration from:

- premium broker apps
- modern fintech platforms
- clean investment dashboards
- Certus Finance-level polish

Design Standards

- smooth transitions
- clean spacing
- readable typography
- polished cards
- balanced chart layouts
- minimal visual noise

---

9. COLOR STRATEGY

Trading Colors

- green for gains
- red for losses

Theme Support

- light mode default
- dark mode supported
- user preference remembered

---

10. LANDING PAGE ARCHITECTURE

Landing page should:

- feel premium
- feel modern
- convert users efficiently
- explain platform intelligently
- highlight AI features naturally

Main Sections

- hero section
- realtime market visuals
- AI intelligence showcase
- feature sections
- learning system showcase
- testimonials later
- CTA sections

---

11. ONBOARDING SYSTEM

Signup Flow

Step 1

Create account.

Step 2

Fill onboarding questionnaire.

Questionnaire determines:

- investing knowledge
- experience level
- learning preference
- UI complexity level

Step 3

Adaptive dashboard generation.

---

12. ADAPTIVE DASHBOARD SYSTEM

Dashboard should adapt based on:

- user level
- onboarding data
- usage behavior
- enabled features

---

13. AI POSITIONING SYSTEM

AI can exist as:

- floating assistant
- contextual side panel
- smart alerts
- inline intelligence widgets

Users can later configure placement.

---

14. AI ARCHITECTURE

AI Provider

OpenRouter

AI Philosophy

AI MUST:

- remain contextual
- understand platform state
- understand viewed stocks
- understand user portfolio
- understand news context

AI MUST NOT:

- behave generically
- hallucinate financial data
- guarantee profits
- create fake certainty

---

15. NEWS INTELLIGENCE SYSTEM

Core AI Flow:

```
News
↓
AI Analysis
↓
Affected Sectors
↓
Impacted Stocks
↓
Educational Explanation
```

---

16. REALTIME MARKET SYSTEM

Market Provider

TwelveData

Architecture Requirements

- backend-only provider access
- Redis-heavy caching
- websocket broadcasting
- provider abstraction layer
- optimized API usage

---

17. WEBSOCKET ARCHITECTURE

Used For:

- live stock prices
- portfolio updates
- smart alerts
- watchlist synchronization
- market updates

Technology

Socket.IO

---

18. CACHE ARCHITECTURE

Cache Layer

Redis

Redis Responsibilities

- market caching
- websocket coordination
- AI response caching
- session optimization
- temporary realtime state

---

19. BACKEND ARCHITECTURE

Backend Stack

- NestJS
- TypeScript
- Prisma ORM
- MySQL
- Redis
- Socket.IO

Architecture Style

Modular Monolith

Chosen because it provides:

- clean scaling
- easier maintenance
- beginner-manageable structure
- future microservice readiness

---

20. BACKEND MODULES

Core Modules

- auth
- users
- onboarding
- portfolio
- trading
- market
- watchlist
- AI
- notifications
- websocket
- admin

---

21. DATABASE ARCHITECTURE

Database

MySQL

ORM

Prisma ORM

Database Goals

- relational consistency
- transaction safety
- scalable schema design
- maintainable relationships

---

22. SECURITY ARCHITECTURE

Mandatory Security Systems

- JWT auth
- refresh tokens
- HTTP-only cookies
- DTO validation
- rate limiting
- backend-only secrets
- websocket guards
- secure environment handling

NEVER

- expose API keys
- trust frontend validation
- bypass auth systems
- hardcode secrets

---

23. DEPLOYMENT ARCHITECTURE

Frontend

Vercel

Backend

Railway or Render

Database

MySQL

Cache

Redis

---

24. DEVOPS PHILOSOPHY

Infrastructure should remain:

- simple initially
- scalable later
- production-aware
- maintainable

---

25. ENVIRONMENT MANAGEMENT

All environment variables MUST:

- remain backend-only
- remain centralized
- never be hardcoded
- remain documented

Critical APIs

- TwelveData
- OpenRouter
- Redis
- JWT secrets

---

26. ENGINEERING PRIORITIES

Priority order:

```
1. Backend correctness
2. Architecture consistency
3. Security
4. Realtime stability
5. Scalability
6. UI polish
7. AI intelligence
```

---

27. PERFORMANCE PHILOSOPHY

Optimize:

- websocket efficiency
- Redis usage
- chart rendering
- dashboard loading
- AI orchestration
- API requests

---

28. TESTING PHILOSOPHY

Test:

- backend reliability
- websocket stability
- AI behavior
- portfolio consistency
- responsive UI
- auth systems

---

29. ENGINEERING RULES (VERY IMPORTANT)

Rule 1

Frontend NEVER directly calls market providers.

Rule 2

All provider logic MUST remain abstracted.

Rule 3

AI systems MUST remain contextual.

Rule 4

Redis caching MUST be heavily used.

Rule 5

Backend architecture MUST remain modular.

Rule 6

UI should feel realistic and professional.

Rule 7

Beginner education should feel integrated naturally.

Rule 8

Advanced users should move faster with less friction.

Rule 9

No fake or unsafe financial certainty.

Rule 10

Always preserve scalability.

---

30. FINAL ARCHITECTURE SUMMARY

YourBuy is designed as:

- a realistic investing simulator
- a premium fintech-style platform
- an AI-powered market intelligence system
- a scalable modular architecture
- a realtime market ecosystem
- a professional portfolio management platform

The system prioritizes:

- backend stability
- contextual AI
- adaptive UX
- realtime architecture
- scalable engineering
- professional product quality

while avoiding:

- tutorial-style UX
- backend chaos
- provider lock-in
- messy architecture
- insecure implementations
- unrealistic financial guidance

END OF MASTER_ARCHITECTURE.md